DSC 395T Algorithms and Data Structures	Individual Project
Flux Equilibrium Clustering	Programming Assignment #5

In this assignment, you will **work individually** to implement the FEC clustering algorithm. In addition to the Python standard library and Pytest, you will be allowed to use NumPy, as one goal of this project is to give you experience with NumPy. You should use NumPy version 2.2.6 with Python 3.10.14.

1 Overview

Clustering algorithms accept as input a set of points in some n-dimensional space and produce as output a set of clusters, so that all points within a cluster are closely related by some metric. For example, a k-means clustering would place the points into one of k clusters, such that all of the points in a given cluster are geometrically closer to each other than to any point in any other cluster.

In this assignment, you will implement Flux Equilibrium Clustering (FEC), a density-seeking graph-based clustering algorithm. Each point in the dataset is treated as a source of flux (e.g., water), which flows downhill on a *k-nearest-neighbor (k-NN) graph* toward the global centroid. As the algorithm iterations, the flux redistributes until an equilibrium is reached. Points that accumulate as sinks become the centroids of clusters.

The input is an $n \times m$ array, where n is the number of points and m is the dimension of the space. The output is an $n \times 1$ array where each element is the cluster number for that point, so the number of unique values is the number of clusters.

2 Parameter Definitions

The following definitions will be useful.

- k: Number of nearest neighbors for graph construction (default: 10)
- σ: Bandwidth parameter for Gaussian kernel weights (default: $0.2\times$ median pairwise distance)
- α: Fraction of stored flux redistributed per iteration (default: 0.5)
- T: Number of transport iterations (default: 25)
- ϵ: Outgoing flux threshold for sink detection (default: 1^{-4})
- β: Reinforcement parameter for flux-aware weights (default: None, uses $\log(1 + f)$)
- $d(i)$: Distance from point x_i to global centroid c, i.e., $d(i) = \|x_i - c\|$, where $\|v\|$ is the Euclidean norm of a vector v.
- c: Global centroid of all data points, $c = \frac{1}{n}\sum_{i=1}^{n} x_i$

3 Algorithm Steps

Step 1. Pre-compute:

- Compute the global centroid $c = \frac{1}{n}\sum_{i=1}^{n} x_i$.
- Calculate the distance from each point to the centroid: $d(i) = \|x_i - c\|$.
- If σ is not specified as a program input, set $\sigma = 0.2 \times \text{median}(\{\|x_i - x_j\| : i < j\})$.
- Set the effective value of k: $k_{\text{eff}} = \min(k, n - 1)$ neighbors.
- Build a symmetric k-NN graph with edge weights using exponential:

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{\sigma^2}\right)$$

Step 2. Initialize Flux:

- Set initial stored flux $f_i(0) = 1$ for each node i.

Step 3. Simulate flow:

- For T iterations, each node i sends flux to neighbors j that satisfy the following relaxed downhill criterion:

$$j \text{ is downhill from } i \text{ if } d(j) \leq d(i) + \tau$$

  where $\tau = 0.1 \times \sigma$ is a tolerance parameter.
- Compute flux-aware edge weights (for $t > 0$):

$$\tilde{w}_{ij}(t) = w_{ij} \cdot \phi(f_j(t))$$

$$\text{where } \phi(f) = \begin{cases} 1 & \text{if } t = 0 \text{ or } \beta = 0 \\ \log(1+f) & \text{if } t > 0 \text{ and } \beta = \text{None} \\ f^{\beta} & \text{if } t > 0 \text{ and } \beta \neq 0 \end{cases}$$

- Send flux along each edge:

$$\delta_{ij}(t) = \alpha \cdot \tilde{w}_{ij}(t) \cdot \frac{f_i(t)}{\sum_{m:d(m)\leq d(i)+\tau} \tilde{w}_{im}(t)}$$

- Update flux:

$$f_i(t+1) = f_i(t) - \sum_j \delta_{ij}(t) + \sum_h \delta_{hi}(t)$$

Step 4. Detect Sinks:

- A vertex i is a sink if its outgoing flux in the final iteration satisfies the following constraint:

$$\sum_j \delta_{ij}(T) < \epsilon$$

- If no sinks are detected, designate the point closest to the centroid as a sink.

Step 5. Assign Clusters with Cycle Detection:

- For each sink s, assign cluster label $\ell(s)$.
- For each non-sink node i, determine the steepest downhill neighbor:

$$\text{steepest}(i) = argmax_{j:d(j)\leq d(i)+\tau} \delta_{ij}(T)$$

- Trace the steepest path to a sink with cycle detection:
  - Follow path $i \to \text{steepest}(i) \to \text{steepest}(\text{steepest}(i)) \to \ldots$
  - If a cycle (closed path) is detected, break it by designating the current node as a new sink
  - Assign all nodes in the path the same cluster label as the terminal sink

Step 6. Post-Processing: Small Cluster Merging:

- Define the minimum cluster size: $s_{\min} = \max(2, \lfloor n/20 \rfloor)$
- For each cluster C with $|C| < s_{\min}$:
  - Compute cluster centroid: $\mu_C = \frac{1}{|C|} \sum_{x \in C} x$
  - Find nearest larger cluster: $C^* = argmin_{C':|C'|\geq s_{\min}} \|\mu_C - \mu_{C'}\|$
  - Merge C into C^*

4 Your Assignment

You will implement the following methods in the module **fec.py**:

```
class FluxEquilibriumClustering:
    def __init__(self, ...) -> None:
    def fit(self, X: np.ndarray) -> "FluxEquilibriumClustering":
```

The `__init__` method stores parameters as instance attributes.

The `fit` method computes the cluster labels following the steps above.

You may ignore case where n=0 or add validation to avoid such a case.

5 Example

This section provides a concrete example illustrating how Flux Equilibrium Clustering (FEC) operates step by step on a small dataset. The purpose of this example is to clarify the mechanics of flux transport, sink detection, and cluster assignment.

5.1 Setup

We consider a dataset of $n = 8$ points in R^2 with the following parameters:

- $k = 2$
- $T = 5$
- $\alpha = 0.7$
- $\epsilon = 10^{-5}$
- $\beta = 0$
- Tolerance parameter $\tau = 0.1 \times \sigma$ (as defined earlier)

The input data matrix is:

$$X = \begin{bmatrix} 2.67 & 2.95 \\ 1.29 & 4.31 \\ 1.57 & 2.33 \\ 3.40 & 2.00 \\ -3.85 & -3.73 \\ -2.56 & -1.17 \\ -2.97 & -3.89 \\ -2.57 & -3.48 \end{bmatrix}$$

5.2 Step 1: Pre-compute

The global centroid is:

$$c = \frac{1}{8} \sum_{i=1}^{8} x_i = (-0.3775, -0.085)$$

The distances of each point to the centroid are:

$$d = \begin{bmatrix} 4.3009 & 4.7007 & 3.1024 & 4.3147 & 5.0343 & 2.4373 & 4.6042 & 4.0414 \end{bmatrix}$$

The pairwise squared distances $\|x_i - x_j\|^2$ are:

$$\begin{bmatrix} 0 & 3.754 & 1.5944 & 1.4354 & 87.1328 & 44.3273 & 78.5952 & 68.8025 \\ 3.754 & 0 & 3.9988 & 9.7882 & 91.0612 & 44.8529 & 85.3876 & 75.5837 \\ 1.5944 & 3.9988 & 0 & 3.4578 & 66.1 & 29.3069 & 59.3 & 50.8957 \\ 1.4354 & 9.7882 & 3.4578 & 0 & 85.3954 & 45.5705 & 75.269 & 65.6713 \\ 87.1328 & 91.0612 & 66.1 & 85.3954 & 0 & 8.2177 & 0.8 & 1.7009 \\ 44.3273 & 44.8529 & 29.3069 & 45.5705 & 8.2177 & 0 & 7.5665 & 5.3362 \\ 78.5952 & 85.3876 & 59.3 & 75.269 & 0.8 & 7.5665 & 0 & 0.3281 \\ 68.8025 & 75.5837 & 50.8957 & 65.6713 & 1.7009 & 5.3362 & 0.3281 & 0 \end{bmatrix}$$

From above, the two nearest neighbors of sample 0 are samples 3 and 2.
The kernel bandwidth is computed as:

$$\sigma = 0.2 \times \text{median}(\{\|x_i - x_j\| : i < j\}), \quad \sigma^2 = 1.7836$$

Since $k = 2$ and $n - 1 = 7$, the effective number of neighbors is $k_{\text{eff}} = 2$.
Using the Gaussian kernel, the resulting k-NN edge weights for each node are:

$$w_{ij} = \begin{bmatrix} 0.4472 & 0.4090 \\ 0.1219 & 0.1062 \\ 0.4090 & 0.1439 \\ 0.4472 & 0.1439 \\ 0.6386 & 0.3853 \\ 0.0502 & 0.0144 \\ 0.8320 & 0.6386 \\ 0.8320 & 0.3853 \end{bmatrix}$$

5.3 Step 2: Initialize Flux

All nodes begin with unit stored flux:

$$f(0) = [1, 1, 1, 1, 1, 1, 1, 1]$$

5.4 Step 3: Simulate Flow

At iteration $t = 0$, $\phi(f) = 1$ for all nodes. phi: [1., 1., 1., 1., 1., 1., 1., 1.] Consider sample 0 as an illustration.

Its downhill neighbors satisfy $d(j) \leq d(0) + \tau$, which includes samples 3 and 2. The corresponding weights are $[0.4472, 0.4090]$ with sum 0.8562. The outgoing flux from sample 0 is distributed as:

$$\delta_{0j}(0) = 0.7 \times \frac{[0.4472, 0.4090]}{0.8562} = [0.3656, 0.3344]$$

Thus, sample 0 sends a total of 0.7 units of flux to its neighbors. Repeating this procedure for all samples yields the following flux values after iteration 0:

$$f(1) = [1.2036, 0.3000, 1.8308, 0.6656, 0.3000, 1.0000, 0.7366, 1.9634]$$

The delta values at the end of iteration 0 are:

$$\delta(0) = \begin{bmatrix} 0.3656 & 0.3344 \\ 0.3740 & 0.3260 \\ 0 & 0 \\ 0.5296 & 0.1704 \\ 0.4366 & 0.2634 \\ 0 & 0 \\ 0.7000 & 0 \\ 0 & 0 \end{bmatrix}$$

After iteration 1, the stored flux becomes:

$$f(2) = [0.8258, 0.0900, 2.4446, 0.6397, 0.0900, 1.0000, 0.3519, 2.5581]$$

After completing all $T = 5$ iterations, the total outgoing flux from each node is:

$$[0.3204, 0.0057, 0, 0.2624, 0.0057, 0, 0.0387, 0]$$

5.5 Step 4: Detect Sinks

δ(T) < ϵ Using the threshold $\epsilon = 10^{-5}$, samples 2, 5, and 7 are identified as sinks.

5.6 Step 5: Assign Clusters with Cycle Detection

Each sink is assigned a unique cluster label:

$$\ell(2) = 0, \quad \ell(5) = 1, \quad \ell(7) = 2$$

Just named the 3 sinks as 0, 1 and 2. It can be any name/label.

For each non-sink node, determine the steepest downhill neighbor based on the largest outgoing flux. For example, sample 0 has outgoing flux $[0.1673, 0.1530]$, so it selects sample 3 as its steepest neighbor. steepest [0]: 3

Likewise, for sample 1, steepest [1]: 0, steepest [3]: 0, steepest [4]: 6, steepest [6]: 7

Tracing paths for sample 0 reveals a cycle $0 \to 3 \to 0$. Per the handout, If a cycle (closed path) is detected, break it by designating the current node as a new sink. So, this cycle is broken by designating sample 3 as a new sink.

labels before [-1 -1 0 -1 -1 1 -1 2] labels after [3 -1 0 -1 -1 1 -1 2]

Likewise, **Tracking for sample 1**: -> 0 Labels after [3 3 0 3 -1 1 -1 2]

Tracking for sample 4: -> 6 -> 7 (Assign all nodes in the path the same cluster label as the terminal sink)

Labels before [3 3 0 3 -1 1 -1 2] labels after [3 3 0 3 2 1 2 2]

After resolving all paths, the cluster labels are:

$$[3, 3, 0, 3, 2, 1, 2, 2]$$

5.7 Step 6: Post-Processing

Small clusters are merged according to the post-processing rule.

labels before:

$$[3 \;\; 3 \;\; 0 \;\; 3 \;\; 2 \;\; 1 \;\; 2 \;\; 2]$$

small clusters are marked red. After merging, the final cluster assignments are:

$$[3, 3, 3, 3, 2, 2, 2, 2]$$

This example demonstrates how flux redistribution, sink detection, cycle handling, and post-processing jointly determine the final clustering.

6 What to Turn in

Use the **Gradescope** page of the **Canvas** website to submit your repository via a linked Github account. Please refer to the "Submission Instructions" in Canvas for Gradescope submission instructions. Canvas is available at `http://canvas.utexas.edu/`.

Important Details

- This assignment is due at 11:59pm on the due date. Late assignments will be penalized 20% per day (unless your Slip days are assigned to it at the end of the semester).
- This is an individual assignment and you are not allowed to work in a group.
- We provide grade feedback upon each submission and **limit 5 submissions**. Canvas and Gradescope cannot enforce the limit as of now, so we ask you not to make more than 5 submissions. If you did, we will pick one from the first 5 submissions i.e. ignore the later ones and may apply some penalty.

Acknowledgments We thank Omkar Bajaj for creating this assignment. And we thank Saffron Liu, Joseph Zhu, Ajay Chhawacharia and Pablo Esparza for their helpful comments on this assignment.